Reviewer Pack — Minimal Brauer Group Rank Correlation with Communication Com…

Entry 398c3d5bff34 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 05:09:59 UTC

Minimal Brauer Group Rank Correlation with Communication Complexity

Entry ID: `398c3d5bff34` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 05:09:59 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory (specifically, Brauer Groups) **Field B** (complexity object): Communication Complexity

Statement:

> For every k-party communication protocol computing a function f: {0,...,n}^k -> {0,...,m}, the minimal rank of the Brauer group Br(Q_l(f)), where Q_l(f) is the quotient ring of the free algebra generated by variables x_{i,j} (for 1 <= i <= k and 1 <= j <= n), modulo the ideal generated by relations encoding f, satisfies |Br(Q_l(f))| = Ω(n^(k/2)).

Rationale (proposer's reasoning):

> The Brauer group provides a robust algebraic invariant for studying field extensions, which may capture hidden structures in communication complexity. A higher rank suggests more complex interactions between parties, potentially implying lower bounds on communication complexity.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e1f76d2bce0b34a1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 80% of k-party protocols, the ratio of the minimal Brauer group rank to n^(k/2) is greater than or equal to 0.5 and the average communication complexity across all seeds does not exceed a threshold determined by bootstrapping with 100 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'algebraic K-theory' AND 'Brauer group rank' AND 'communication complexity' - 'Brauer group' inproceedings arxiv:alg-geom OR arxiv:math.AG AND 'communication complexity' - 'communication protocol' AND 'function computation' AND 'quotient ring' AND 'Brauer group'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda x: abs(A[x][i]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            denom = A[i][i]
            for j in range(n):
                A[i][j] /= denom
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_rank(A):
        rank = 0
        A = gaussian_elimination(A)
        for row in A:
            if any(row):
                rank += 1
        return rank

    def generate_quotient_ring(k, n):
```

```
        variables = [[f"x_{i}_{j}" for j in range(n)] for i in range(k)]
        relations = []
        # Example relation: x_0_0 * x_1_0 - x_2_0
        relations.append([variables[0][0], variables[1][0], -variables[2][0]])
        return variables, relations

    def compute_brauer_group_rank(k, n):
        variables, relations = generate_quotient_ring(k, n)
        A = [[0] * (n + k) for _ in range(n + k)]
        for i in range(n):
            A[i][i] = 1
        for relation in relations:
            for j in range(len(relation)):
                if relation[j] != 0:
                    for l in range(len(variables[0])):
                        if variables[l][j] in relation:
                            A[i][l + n] += relation[j]
                            break
        return matrix_rank(A)

    def communication_complexity(k, n):
        # Example complexity: k * n bits
        return k * n

    k = random.randint(2, 5)
    n = random.randint(10, 30)
    instances_tested = 30
    total_brauer_rank = 0
    total_communication_complexity = 0

    for _ in range(instances_tested):
        brauer_rank = compute_brauer_group_rank(k, n)
        communication_comp = communication_complexity(k, n)
        total_brauer_rank += brauer_rank
        total_communication_complexity += communication_comp

    mean_brauer_rank = total_brauer_rank / instances_tested
    avg_communication_complexity = total_communication_complexity / instances_tested
    conjecture_holds = (mean_brauer_rank >= 0.5 * n ** (k / 2)) and (avg_communication_complexity

    return {
        "metric_name": "Brauer group rank",
        "metric_value": mean_brauer_rank,
        "instances_tested": instances_tested,
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"mean_brauer_rank={mean_brauer_rank}, avg_c
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {result}")
        results.append(result)

    mean_brauer_rank = sum(r["metric_value"] for r in results) / len(results)
    std_deviation = math.sqrt(sum((r["metric_value"] - mean_brauer_rank) ** 2 for r in results) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_brauer_rank} std={std_deviation} support_fraction={su
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_brauer_rank} std={std_deviation} support_fraction={su
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab71a0f8.py", line 102, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab71a0f8.py", line 78, in run_trial
    brauer_rank = compute_brauer_group_rank(k, n)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab71a0f8.py", line 54, in compute_braue
    variables, relations = generate_quotient_ring(k, n)
                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ab71a0f8.py", line 50, in generate_quot
    relations.append([variables[0][0], variables[1][0], -variables[2][0]])
                                                         ^^^^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18636
2	propose	ollama_remote	glm4:latest	0	0	13622
3	propose	ollama_remote	glm4:latest	0	0	17742
4	preregistration	ollama_remote	glm4:latest	0	0	9655
5	novelty	ollama_remote	glm4:latest	0	0	8604
6	novelty	ollama_remote	glm4:latest	0	0	8989
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20664
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10354
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10643
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27370
11	judge	ollama_remote	glm4:latest	0	0	8873

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 155152 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/398c3d5bff34.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/398c3d5bff34.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/398c3d5bff34.tar.gz` (if generated)